

sickles, and knuts. The reason is that all methods have a first parameter named `self`. When a method is called on an object, the object is automatically passed in for the `self` parameter. The rest of the arguments are assigned to parameters normally. If you see an error message, such as `TypeError: __init__() takes 3 positional arguments but 4 were given`, you've probably forgotten to add the `self` parameter to the method's `def` statement.

You don't have to name a method's first parameter `self`; you can name it anything. But using `self` is conventional, and choosing a different name will make your code less readable to other Python programmers. When you're reading code, the presence of `self` as the first parameter is the quickest way you can distinguish methods from functions. Similarly, if your method's code never needs to use the `self` parameter, it's a sign that your method should probably just be a function.

The 2, 5, and 99 arguments of `WizCoin(2, 5, 99)` aren't automatically assigned to the new object's attributes; we need the three assignment statements in `__init__()` to do this. Often, the `__init__()` parameters are named the same as the attributes, but the presence of `self` in `self.galleons` indicates that it's an attribute of the object, whereas `galleons` is a parameter. This storing of the constructor's arguments in the object's attributes is a common task for a class's `__init__()` method. The `datetime.date()` call in the previous section did a similar task except the three arguments we passed were for the newly created `date` object's `year`, `month`, and `day` attributes.

You've previously called the `int()`, `str()`, `float()`, and `bool()` functions to convert between data types, such as `str(3.1415)` returning the string value `'3.1415'` based on the float value `3.1415`. Previously, we described these as functions, but `int`, `str`, `float`, and `bool` are actually classes, and the `int()`, `str()`, `float()`, and `bool()` functions are constructor functions that return new integer, string, float, and Boolean objects. Python's style guide recommends using capitalized camelcase for your class names (like `WizCoin`), although many of Python's built-in classes don't follow this convention.

Note that calling the `WizCoin()` construction function returns the new `WizCoin` object, but the `__init__()` method never has a `return` statement with a return value. Adding a return value causes this error: `TypeError: __init__() should return None`.